A

Mini Project Report

on

"BITTRUTH-To Verify Authenticity of Media"

Submitted in partial fulfillment of the requirements for the

Degree

Third Year Engineering – Computer Science and Engineering (Data Science)

By

SHRIKANT THAKUR	**23107045**
ARJUN TALEKAR	**23107005**
AAYUSH THAKUR	**23107015**
PRANAV PARAB	**23107049**

Under the guidance of

Prof. SWATI PARHAD

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING (DATA SCIENCE)

A.P. SHAH INSTITUTE OF TECHNOLOGY

G.B. Road, Kasarvadavali, Thane (W)-400615

UNIVERSITY OF MUMBAI

Academic year: 2025 - 2026

CERTIFICATE

This to certify that the Mini Project report on **"BITTRUTH-To Verify Authenticity of Media"** has been submitted by **Shrikant Thakur (23107045)**, **Arjun Talekar (23107005)** and **Aayush Thakur (23107015) and Pranav Parab (23107049)** who are bonafide students of A. P. Shah Institute of Technology, Thane as a partial fulfillment of the requirement for the degree in **Computer Science Engineering (Data Science)**, during the academic year **2025-2026** in the satisfactory manner as per the curriculum laid down by University of Mumbai.

Prof. Swati Parhad
Guide

Dr. Pravin Adivarekar
HOD, CSE(Data Science)

Dr. Uttam D. Kolekar
Principal

External Examiner:
1.

Internal Examiner:
1.

Place: A. P. Shah Institute of Technology, Thane
Date:

ACKNOWLEDGEMENT

This project would not have come to fruition without the invaluable help of our guide **Prof. Swati Parhad** Expressing gratitude towards our HoD, **Dr. Pravin Adivarekar**, and the Department of Computer Science Engineering (Data Science) for providing us with the opportunity as well as the support required to pursue this project. We would also like to thank our project coordinator **Ms. Shubhangi Soni** and **Ms. Avani N** who gave us her valuable suggestions and ideas when we were in need of them. We would also like to thank our peers for their helpful suggestions.

TABLE OF CONTENTS

ABSTRACT

BitTruth is a machine learning-based media verification system designed to detect potential deepfake manipulations in images and videos, addressing the growing challenge of misinformation in the digital era. With the rapid advancement of synthetic media generation techniques, distinguishing between authentic and manipulated content has become increasingly difficult, leading to reduced trust in digital information.

The system is developed as a backend-first, modular prototype that analyzes uploaded media using feature extraction techniques and a trainable classification pipeline to identify inconsistencies. BitTruth provides users with a confidence score, a classification verdict (such as *likely_real*, *suspicious*, or *likely_fake*), and a concise explanation to enhance interpretability and usability.

The architecture follows a scalable and extensible design, implemented using Python and FastAPI, with OpenCV and NumPy handling core processing tasks. The system supports multiple datasets through independent training pipelines, where each dataset is processed using separate scripts and produces dedicated trained model files. This modular design allows flexible model switching and easier future enhancements. Training can be performed on standard systems with optional NVIDIA GPU acceleration, ensuring adaptability for different computational environments.

Currently deployed as a local prototype with a basic user interface for media upload, BitTruth is designed with future expansion in mind, including integration with a React-based frontend and cloud deployment. The platform aims to provide a simple yet effective tool for preliminary deepfake detection, contributing to digital media authenticity awareness and serving as a foundation for more advanced forensic systems.

Keywords — BitTruth, Deepfake Detection, Media Authenticity, Machine Learning, OpenCV, FastAPI, Feature Extraction, Classification Pipeline, AI Verification, Digital Trust.

.

Chapter 1

Introduction

In today's rapidly evolving digital landscape, the authenticity of visual media has become increasingly difficult to verify. The rise of deepfake technologies and advanced image/video manipulation tools has made it possible to create highly realistic but misleading content. As a result, individuals, organizations, and institutions face growing challenges in distinguishing genuine media from manipulated content, leading to misinformation, loss of trust, and potential misuse across social, political, and digital platforms.

Existing deepfake detection solutions are often complex, resource-intensive, or designed primarily for research and forensic applications, making them less accessible for general users or basic verification needs. Additionally, many available tools lack transparency in their outputs, offering limited explanation or interpretability for the end user.

BitTruth is developed as a lightweight, backend-first media verification system aimed at addressing these challenges through a simple, modular, and user-friendly approach. The platform focuses on detecting potential manipulation in images and videos using machine learning-based analysis. By applying feature extraction techniques and classification models, the system evaluates uploaded media and generates a confidence score, a classification and a concise explanation to help users understand the result.

The system is designed with a modular architecture, where multiple datasets are handled independently through separate training pipelines and dedicated model files. This approach enables flexibility in model selection and future scalability. BitTruth is implemented using Python and FastAPI for backend operations, with OpenCV and NumPy supporting core media processing. The system supports training on standard Windows environments with optional NVIDIA GPU acceleration, allowing adaptability based on available computational resources.

This report presents a comprehensive overview of BitTruth, including its system architecture, machine learning pipeline, modular design approach, dataset handling strategy, and implementation details. By combining machine learning techniques with a focus on usability and scalability, BitTruth aims to contribute toward improving digital trust and awareness in the era of synthetic media.

1.1 Purpose:

The purpose of developing BitTruth is to create a unified and accessible system that simplifies the detection of manipulated digital media, particularly deepfake images and videos. With the increasing use of synthetic media technologies, it has become essential to provide users with a reliable method to verify the authenticity of digital content without requiring advanced technical expertise.

- BitTruth is designed to analyze uploaded images and videos using machine learning-based techniques, enabling users to identify potential signs of manipulation. The system provides a confidence score along with a classification verdict such as likely_real, suspicious, or likely_fake, helping users make informed judgments about the authenticity of media content.
- Another key purpose of the project is to promote awareness regarding digital misinformation and the risks associated with deepfake technologies. By offering simple explanations alongside detection results, the system enhances interpretability and usability for non-technical users.
- Additionally, the project aims to develop a modular and extensible prototype that supports multiple datasets, separate training pipelines, and flexible model switching. This design approach allows future improvements and integration with advanced machine learning models, frontend interfaces, and cloud-based deployment.

1.2 Problem Statement:

In the current digital environment, the rapid advancement of deepfake and media manipulation technologies has made it increasingly difficult to distinguish between authentic and fabricated content. High-quality manipulated images and videos can be generated using accessible tools, leading to the spread of misinformation, fake news, and deceptive digital content.

- Many users lack the technical knowledge or tools required to verify the authenticity of digital media. As a result, they may unknowingly trust or share manipulated content, which can have serious social, political, and ethical consequences.
- Existing deepfake detection systems are often complex, resource-intensive, or designed for specialized research purposes. These solutions may not be easily accessible to general users and often do not provide clear explanations or user-friendly outputs.
- Therefore, there is a need for a lightweight, secure, and user-friendly solution that can perform basic deepfake detection, provide interpretable outputs, and serve as a foundation for future advancement

1.3 Objectives:

The primary objective of **BitTruth** is to develop a simple, intelligent, and user-centric system for detecting manipulated digital media, particularly deepfake images and videos. The project aims to utilize machine learning techniques and modern backend technologies to enhance digital trust, improve awareness of media authenticity, and provide accessible verification tools. By integrating media analysis, classification, and explainable outputs into a unified platform, the system seeks to assist users in identifying potentially misleading content. The key objectives of the project are as follows:

1. To design a machine learning-based media verification system:

The first objective is to develop a system capable of analyzing images and videos to detect potential signs of manipulation using machine learning techniques. The system employs feature extraction methods to identify inconsistencies in media content and uses a classification pipeline to determine whether the media is authentic or fake. This enables users to perform basic verification without requiring advanced technical knowledge.

2. To provide interpretable and user-friendly detection results:

The second objective is to design an output mechanism that presents results in an understandable format. The system generates a confidence score along with a classification verdict such as *likely_real*, *suspicious*, or *likely_fake*. Additionally, it provides a short explanation highlighting possible reasons for the classification, helping users better understand the outcome and increasing transparency in decision-making.

3. To develop a modular and extensible architecture:

The third objective is to build the system using a modular design approach where multiple datasets are handled independently through separate training scripts and dedicated model files. This allows flexible model switching, easier maintenance, and scalability for future improvements. The architecture is designed to support enhancements such as integration with advanced models, frontend applications, and cloud-based deployment.

4. To ensure flexibility in training and deployment environments:

The fourth objective is to enable the system to be trained and executed in different computational environments. The project supports training on standard systems with CPU as well as optional acceleration using NVIDIA GPUs on Windows machines. This flexibility ensures that the system can handle varying dataset sizes and computational requirements.

5. To create a foundation for future deepfake detection advancements:

The final objective is to develop BitTruth as a foundational prototype that can be extended into a more advanced system. Future enhancements may include real-time video analysis, integration of deep learning models such as CNNs or transformers, cloud deployment, and improved user interfaces. The system is intended to serve as a stepping stone toward more robust media authentication solutions.

1.4 Scope:

The scope of **BitTruth** extends across various user groups and application areas that require verification of digital media authenticity. The system is designed as a lightweight, user-centric platform that assists individuals in identifying potential deepfake manipulations in images and videos. By providing machine learning-based analysis, confidence scoring, and interpretable outputs, BitTruth serves as a practical tool for basic media verification and awareness. The following are the primary user groups within the scope of this project:

1. General Users:

Individuals who frequently consume digital content on social media and online platforms can use BitTruth to verify the authenticity of images and videos before trusting or sharing them. The system helps users identify potentially manipulated media and promotes responsible content consumption.

2. Students and Researchers:

Students and academic researchers can utilize the platform as a learning tool to understand how deepfake detection systems work. It provides a practical demonstration of machine learning concepts such as feature extraction, classification, and media analysis, making it suitable for educational purposes.

3. Content Creators and Media Professionals:

Content creators, journalists, and media professionals can use BitTruth as a preliminary verification tool to assess the authenticity of visual content before publishing or sharing it. This helps in reducing the risk of spreading misleading or manipulated media.

4. Developers and AI Enthusiasts:

Developers interested in artificial intelligence and computer vision can explore the modular architecture of BitTruth. The system supports multiple datasets, separate training pipelines, and independent model files, allowing experimentation, customization, and further development.

5. Educational and Institutional Use:

Educational institutions can use BitTruth as a demonstration project for teaching concepts related to machine learning, computer vision, and digital forensics. It serves as a foundational prototype for understanding real-world applications of AI in media verification.

6. Prototype and Research Applications:

As a mini-project, BitTruth is primarily intended for academic and experimental purposes. It provides a base system that can be extended into more advanced solutions involving deep learning models, real-time video analysis, and cloud-based deployment.

Chapter 2

Literature Review

2.1 Challenges in Digital Media Authenticity and Misinformation:

Research in the domain of digital media highlights that the rapid growth of social media platforms and content-sharing applications has significantly increased the spread of manipulated and misleading content. Studies indicate that users often lack the necessary tools and awareness to verify the authenticity of images and videos, leading to the unintentional spread of misinformation.

The accessibility of deepfake generation tools has further intensified this issue, allowing even non-experts to create realistic manipulated media. While awareness about misinformation has improved, there remains a gap in simple, user-friendly verification systems. BitTruth addresses this gap by providing an accessible platform for basic media authenticity verification.

2.2 Deepfake Detection Techniques:

Deepfake detection has been widely studied using machine learning and deep learning approaches. Traditional methods focus on detecting inconsistencies in pixel patterns, lighting conditions, and facial artifacts, while more advanced techniques use Convolutional Neural Networks (CNNs) to identify subtle manipulation traces. Recent research emphasizes hybrid approaches combining feature extraction and classification techniques to balance computational efficiency and detection performance. BitTruth adopts a similar approach by utilizing feature extraction methods along with a trainable classification pipeline, making it suitable for lightweight and prototype-level implementations.

2.3 Computer Vision and Feature Extraction Methods:

Computer vision techniques play a crucial role in analyzing digital media for authenticity. Methods such as edge detection, texture analysis, and frequency-domain analysis are commonly used to identify anomalies in images and video frames. Libraries such as OpenCV have been widely used in academic and industrial applications for preprocessing and feature extraction tasks.

2.4 Machine Learning-Based Classification Systems:

Machine learning classification models have been extensively applied in image and video analysis tasks, including spam detection, object recognition, and anomaly detection. In deepfake detection, classification models are trained on labeled datasets containing real and manipulated media to distinguish between authentic and fake content. Many existing systems rely on large-scale datasets and computationally intensive models, which may not be suitable for lightweight applications. BitTruth focuses on a modular classification approach, where separate models are trained on different datasets and stored independently. This allows flexibility in model selection and easier experimentation while maintaining a manageable system complexity.

2.5 Comparative Analysis of Existing Solution:

Below presents a comparison of existing media authenticity and deepfake detection solutions based on their approach, supported media types, explainability, and accessibility. It highlights that many current platforms focus mainly on video detection and provide limited user-level explanations. In comparison, BitTruth offers support for both image and video analysis while providing clear outputs such as confidence score, verdict, and explanation. The table demonstrates how BitTruth aims to be a lightweight, user-friendly, and more transparent alternative to existing solutions.

Platform	Approach	Media Support	Explainability	Accessibility
Deepware Scanner	Deep Learning (CNN-based)	Video-focused	Limited explanation	Web-based
Sensity AI	Advanced AI Detection Models	Image & Video	Minimal user-level explanation	Enterprise-focused
Microsoft Video Authenticator	AI-based confidence scoring	Video	Confidence score only	Limited availability
BitTruth	Feature Extraction + ML Classification	Image & Video	Confidence + Verdict + Explanation	Lightweight, user-friendly prototype

Table 2.5: Existing Solutions

Chapter 3

Proposed System

BitTruth proposes a machine learning-based media verification system that focuses on detecting potential deepfake manipulations in images and videos through a simple, modular, and user-friendly approach. The system is designed around three guiding principles: **simplicity** (easy media upload and analysis), **interpretability** (clear confidence score, verdict, and explanation), and **modularity**.

3.1 Features and Functionality

1. Media Analysis Engine:-

- The Media Analysis Engine is the core component responsible for processing uploaded images and videos. It performs preprocessing operations such as resizing, normalization, and frame extraction using OpenCV and NumPy to prepare the media for analysis.

- The system extracts visual features such as texture patterns, edge inconsistencies, and structural anomalies that may indicate manipulation. These extracted features are then passed to the classification pipeline for further evaluation.

2. Machine Learning Classification Module:-

- The classification module uses a trainable machine learning pipeline to determine whether the given media is authentic or manipulated. It is trained on multiple datasets, where each dataset is handled through separate training scripts and produces its own model file.
- The system generates outputs in the form of a confidence score along with a verdict such as *likely_real*, *suspicious*, or *likely_fake*, allowing users to quickly understand the authenticity level of the media.

3. Explainable Output System:-

- BitTruth provides an explainable output mechanism that enhances user understanding by presenting a short reasoning along with the result. Instead of only displaying numerical values, the system highlights possible causes such as unusual textures, inconsistent edges, or visual artifacts.

4. Modular Training and Model Management:-

- The system is designed with a modular structure where datasets are stored separately and processed using independent training scripts. Each training process generates a separate trained model file, allowing flexibility in model selection and updates.
- This modular approach enables easy experimentation, scalability, and future integration of more advanced models without affecting the overall system structure.

5. Backend API System:-

- The backend of BitTruth is implemented using FastAPI, which handles media uploads, processing requests, and result generation efficiently. The API structure allows smooth communication between different system components.
- The backend-first approach ensures that the core functionality remains independent of the user interface, making it easier to extend the system in the future.

6. Basic User Interface:-

- A simple and temporary user interface is implemented to allow users to upload images and test the system functionality. The interface focuses on demonstrating the working of the backend system rather than providing a complete user experience.

7. Flexible Deployment Support:-

- BitTruth currently operates as a local prototype system but is designed to support future deployment on cloud platforms. The architecture allows easy scaling and integration with external services.
- The system also supports training on Windows machines with optional NVIDIA GPU acceleration, ensuring flexibility in handling different dataset sizes and computational requirements.

3.2 Architecture and Module Deep Dive:

BitTruth is built on a modular, backend-first architecture that separates user interaction, processing logic, and machine learning computation into distinct components. This separation ensures that media processing and model inference tasks do not affect the responsiveness of the API layer, while also enabling flexibility in extending the system with new models or frontend interfaces.

1. Frontend Layer:-

• The frontend serves as the user interaction layer of the system, allowing users to upload images or videos for analysis. It is implemented as a simple and lightweight interface focused on functionality rather than design complexity, ensuring ease of use during testing and demonstration. The interface provides basic input handling for file uploads and displays results including confidence score, classification verdict, and explanation. The frontend communicates with the backend through REST API calls, ensuring a clear separation between presentation and processing logic.

2. API Layer (FastAPI Backend):-

• The FastAPI backend acts as the central processing layer of the system, handling all incoming requests from the frontend. It manages file uploads, routes requests to the appropriate processing modules, and returns structured responses to the user. The backend ensures efficient handling of media data and integrates seamlessly with the machine learning pipeline. It is designed to be scalable and supports future integration with advanced frontend frameworks such as React as well as deployment on cloud platforms.

3. Media Processing and Machine Learning Layer:-

The machine learning layer forms the core intelligence of BitTruth and is responsible for analyzing media content and generating predictions. When a user uploads an image or video, the system follows the workflow below:

Step 1:- Preprocessing: The uploaded media is processed using OpenCV and NumPy. For images, operations such as resizing and normalization are performed, while for videos, frames are extracted for analysis.

Step 2:- Feature Extraction: The system extracts relevant visual features such as texture patterns, edges, and inconsistencies that may indicate manipulation. These features serve as input to the classification-model.

Step 3:- Classification: The extracted features are passed to a trained machine learning model, which predicts whether the media is authentic or manipulated based on learned patterns from the dataset.

Step 4:- Result Generation: The system generates a structured output including a confidence score, a classification verdict (likely_real, suspicious, or likely_fake), and a short explanation describing the possible reasons behind the result. The response is then sent back to the API layer and displayed to the user.

4. Modular Training and Model Management Module:-

• BitTruth follows a modular training approach where each dataset is stored in a separate directory and processed using an independent training script. Each dataset produces its own trained model file, allowing multiple models to coexist within the system. This design enables flexible model selection, easier updates, and experimentation with different datasets without affecting the overall system. It also allows future integration of more advanced models such as deep learning-based approaches.

5. Dataset Management Module:-

• The dataset module handles the organization and usage of multiple datasets used for training and evaluation. Datasets such as Deepfake and Real Images, DeepDetect 2025, and Deepfake Image Detection are maintained separately to ensure clarity and modularity. Each dataset is preprocessed independently and used to train specific models, enabling better control over data handling and improving the adaptability of the system for different use cases.

Overall, the architecture of BitTruth ensures a clean separation of concerns, efficient processing of media data, and flexibility for future enhancements. The modular design makes the system suitable for academic experimentation while also providing a foundation for scalable and advanced deepfake detection systems.

3.3 Database Schema Design (SQLite):

The SQLite database serves as the persistent storage layer for all structured data in BitTruth, including user inputs, processed media records, model outputs, and system logs. The schema is designed using basic normalization principles to ensure data consistency and efficient retrieval. The lightweight and offline-first nature of SQLite makes it suitable for a prototype system, allowing the application to function locally without requiring a dedicated database server.

- **Table 1: Media Analysis**

The Media_Analysis table is the central data storage component of BitTruth, responsible for maintaining records of all uploaded media files and their corresponding analysis results. Each record is uniquely identified by an auto-incremented integer id, which serves as the primary key for the table. The file_name column stores the name of the uploaded image or video file as a text field, allowing identification of the

media being processed. The file_type column specifies whether the uploaded content is an image or video, enabling the system to apply appropriate preprocessing techniques.

Column Name	Data Type	Description
file_path	TEXT	Stores path of uploaded media file.
upload_timestamp	DATETIME	Stores upload date and time automatically.
confidence_score	FLOAT	Stores model prediction score.
verdict	TEXT	Stores result like real, suspicious, or fake.
explanation	TEXT	Stores short reason for prediction.
model_used	TEXT	Stores name of model used.
processing_time	FLOAT	Stores time taken for analysis.

Table 3.1: Media Analysis

- **Table 2: Sessions**

The Sessions table manages request-level tracking and basic processing sessions within BitTruth, helping maintain a record of media analysis activities performed by the system. Although BitTruth primarily operates as a lightweight prototype without complex user authentication, this table is used to monitor individual analysis requests and maintain traceability of operations.

Column	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique session identifier
media_id	INTEGER	FOREIGN KEY (Media_Analysis.id)	Reference to the processed media record
session_id	TEXT	NOT NULL	Unique identifier for each analysis session
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Session creation timestamp
completed_at	DATETIME	NULL	Timestamp when processing is completed
status	TEXT	DEFAULT 'processing'	Indicates session state (processing, completed)

Table 3.2: Sessions

- **Table 3: Model Results**

The Model_Results table stores all outputs generated by the machine learning models in BitTruth, including analysis results, confidence scores, and explanations for each processed media file.

Column	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique result identifier
media_id	INTEGER	FOREIGN KEY	Reference to the analyzed media
model_name	TEXT	NOT NULL	Name of model used for analysis
confidence_score	REAL	NOT NULL	Confidence score generated by model
Verdict	TEXT	NOT NULL	Output label

Table 3.3: Model Results

- **Table 4: Datasets**

The Datasets table acts as the core data repository for BitTruth, storing metadata related to all datasets used for training and evaluation of the machine learning models. It enables organized dataset management and supports the modular training architecture of the system, where each dataset is handled independently.

Column Name	Data Type	Description
id	INTEGER	Unique ID for each dataset (Primary Key).
dataset_name	TEXT	Stores name of dataset.
description	TEXT	Stores short details about dataset.
data_type	TEXT	Stores type such as image, video, or both.
total_samples	INTEGER	Stores total number of samples.
storage_path	TEXT	Stores dataset folder path.
created_at	DATETIME	Stores creation date and time automatically.

Table 3.4: Datasets

Table 5: Models

The Models table manages all trained machine learning models used in BitTruth, supporting the modular design where each dataset produces a separate model file. This table enables efficient model tracking, selection, and updating within the system.

Column Name	Data Type	Description
id	INTEGER	Unique ID for each model (Primary Key).
model_name	TEXT	Stores model name or identifier.
dataset_id	INTEGER	References dataset used for training.
model_path	TEXT	Stores trained model file path.
accuracy	FLOAT	Stores model accuracy score.
created_at	DATETIME	Stores creation date and time automatically.

Table 3.5: Models

Table 6: System_Logs

The System_Logs table is responsible for maintaining records of system activities, processing events, and errors that occur during the execution of BitTruth. It plays an important role in debugging and monitoring

Column Name	Data Type	Description
id	INTEGER	Unique ID for each log entry (Primary Key).
session_id	INTEGER	Stores related processing session ID.
log_type	TEXT	Stores type such as info, warning, or error.
message	TEXT	Stores event or error details.
created_at	DATETIME	Stores log date and time automatically.

Table 3.6: System Logs

Overall, these tables collectively support the data management, model handling, and system monitoring aspects of BitTruth, ensuring that the application remains organized, traceable, and extensible for future enhancements.

Chapter 4

Requirements Analysis

The requirement analysis for **BitTruth** involves systematically identifying and documenting the necessary functional and non-functional requirements that the system must satisfy to achieve its objective of detecting manipulated digital media. Given the increasing challenges related to deepfake content and misinformation, the system is designed with a focus on simplicity, interpretability, scalability, and reliability. The functional requirements define the core operations and features of the system, while the non-functional requirements address performance, usability, and system constraints. Together, these requirements provide a structured foundation for the design, development, and evaluation of the BitTruth prototype.

4.1 Functional Requirements (FRs):

The following requirements define the specific behaviours and functionalities that the BitTruth system must exhibit to perform effective deepfake detection and media authenticity verification. These requirements map directly to the system's core features and are designed to ensure that users can easily analyze digital media and understand the results through a simple and unified interface.

• The system must allow users to upload images and videos for analysis through a basic user interface.

• The system must preprocess uploaded media using OpenCV and NumPy, including resizing, normalization, and frame extraction for video inputs.

• The system must extract relevant visual features such as textures, edges, and inconsistencies from the media for analysis.

• The system must implement a machine learning-based classification pipeline to determine whether the media is authentic or manipulated.

• The system must generate a confidence score indicating the likelihood of the media being real or fake.

• The system must classify outputs into categories such as likely_real, suspicious, or likely_fake.

• The system must provide a short explanation describing possible reasons behind the classification result.

• The system must support multiple datasets with separate training scripts and independent trained model files.

• The system must allow flexible model usage by enabling switching between different trained models.

• The system must provide a FastAPI-based backend to handle requests, processing, and response delivery.

• The system must support execution on local systems with CPU and optional NVIDIA GPU acceleration for training tasks.

4.2. Non-Functional Requirements (NFRs):

The non-functional requirements govern the operational qualities of **BitTruth**, defining how the system performs rather than what it does. In the context of a deepfake detection system designed for academic and basic verification use, these requirements are equally important as functional ones. These requirements directly address key concerns such as performance, usability, flexibility, and reliability, which are essential for a system handling media processing and machine learning-based analysis.

• **Performance:** Efficient media processing using OpenCV and NumPy; analysis results generated within acceptable time limits depending on media size and system configuration.

• **Scalability:** Modular architecture with separate datasets and model files enables independent updates and extension without affecting the overall system.

• **Usability:** Simple and user-friendly interface for uploading media and viewing results; clear output format including confidence score, verdict, and explanation.

• **Flexibility:** Support for multiple datasets and independent training pipelines; compatibility with both CPU-based and NVIDIA GPU-accelerated environments.

• **Accuracy:** Machine learning classification pipeline designed to identify basic manipulation patterns; results dependent on dataset quality and feature extraction effectiveness.

Chapter 5

Project Design

5.1 Use Case diagram:

The Use Case Diagram captures the primary actors and interactions within **BitTruth**. The primary actors are: the User (general user interacting with the system), the Administrator, and the Machine Learning Model (analysis engine as a system actor). Key use cases include: Upload Media (image/video input), Analyze Media (processing through ML pipeline), View Results (confidence score, verdict, explanation), Select Model (choose trained model), and Manage Dataset/Models (for system updates). The Administrator use cases include: Monitor System Activity, Manage Datasets, Update Models, and Review Analysis Logs.

The Use Case Diagram (Fig 5.1) captures the main functionalities and interactions between the system's actors and the BitTruth platform, focusing on the high-level objectives of media upload, machine learning-based analysis, and result interpretation. This model serves as an essential tool for communicating the system's intended functionality and scope to both technical and non-technical stakeholders.

Actors:

- **User:** Represents the primary user of the system — any individual who wants to verify the authenticity of images or videos. This includes students, researchers, content creators, and general users who interact with the system to analyze media and view results.
- **Administrator:** Represents the authorized user responsible for system management and maintenance. The Administrator has access to additional functionalities such as managing datasets, updating trained models, and monitoring system performance and logs.
- **Machine Learning Model:** Represents the internal system component responsible for analyzing media content and generating predictions. It acts as the core processing entity that performs feature extraction, classification, and result generation based on trained data.

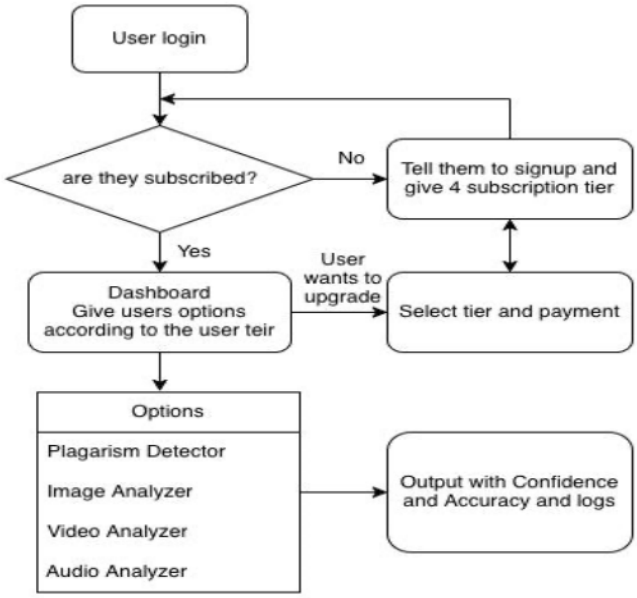


Fig 5.1: Use Case

- **Components of the Use Case Diagram:**

To maintain consistency with your project report, here is the detailed breakdown of the components for the **BitTruth** Use Case Diagram, mapped to your specific technical details and the structure of the reference you provided.

1. **System Boundary — BitTruth Platform:** The rectangular box represents the boundary of the **BitTruth detection system**, enclosing all internal logic handled by the FastAPI backend and clearly separating them from external actors like the User or independent Training Scripts.
2. **Upload Image or Video for Analysis:** This is the primary entry point for the **Citizen User**. It handles the reception of multi-part form data. It triggers the internal processing pipeline and is the first step in the media verification workflow.
3. **Analyze Uploaded Media and Reason:** This core use case represents the logic where the system samples frames (using **OpenCV**) and prepares data for the model. It "includes" the specific classification and reasoning steps to ensure a complete response is generated.
4. **View Content Verification Result:** This component displays the final output to the user. It presents the **Confidence Score** (quantitative) and the **BitTruth Verdict** (qualitative), such as "Likely Real" or "Suspicious," along with a short explanation of the detected inconsistencies.

5. **Select Backend Model:** A crucial modular component that allows the system to choose which trained model file to use for a specific request. This is triggered via an "include" relationship during the analysis phase, ensuring the correct dataset-specific model is active.
6. **Manage Dataset Folders:** Exclusive to the **Administrator/Developer**, this use case handles the organization of separate folders for datasets like *DeepDetect 2025* or *Deepfake Image Detection*. It ensures that data remains isolated for specific training tasks.
7. **Run Dataset Training Script:** This represents the execution of the separate Python training scripts. It "includes" the **Execute Training on CPU/GPU** logic, where the system checks for NVIDIA hardware to accelerate the feature extraction and classification training.
8. **Execute Training on CPU/GPU:** A specialized use case that manages hardware resources. It attempts to utilize **NVIDIA CUDA** for faster processing but contains fallback logic to use the **CPU** if a dedicated GPU is unavailable, ensuring flexibility across different Windows PCs.
9. **Generate Trained Model File:** This is the successful outcome of a training session. It involves saving the trained weights into a model file (e.g., .pkl or .h5) that can later be swapped into the live detection engine.
10. **Monitor System Logs and Performance:** An **Administrator-exclusive** use case that tracks backend performance, inference times, and any errors during the image processing or model switching phases.
11. **ML Classification Pipeline:** This internal component uses **NumPy** for data normalization and trained classification algorithms to determine the likelihood of manipulation. It is "used" by the backend system whenever a file is processed.
12. **Add New Dataset:** This use case allows the **Developer** to extend the system by importing new Kaggle datasets. It is linked to the **Manage Dataset Folders** flow, facilitating the planned future growth of the BitTruth platform.

5.2 DFD (Data Flow Diagram):

- The Data Flow Diagram (DFD) visually represents the flow of media data and processing within BitTruth's modular architecture, illustrating how the system processes user-uploaded images and videos through the FastAPI backend and machine learning pipeline to generate an authenticity verdict and explanation.

- At Level 0, the Context Diagram represents the entire BitTruth system as a single central process. A user uploads an image or video through the frontend interface. The system processes the input through the FastAPI backend and machine learning model, and returns a confidence score, classification verdict, and explanation back to the user. The Administrator interacts with the system to monitor activity, manage datasets, and update models, while the Machine Learning Model exchanges processed features and predictions with the system to generate analysis results.

- At Level 1, the diagram expands into multiple sub-processes. The uploaded media first enters the Media Upload Module (Process 1.0), where it is validated and stored. The data is then passed to the Preprocessing Module (Process 2.0), which performs operations such as resizing, normalization, and frame extraction. The processed data is forwarded to the Feature Extraction Module (Process 3.0), where relevant visual features are identified. These features are then passed to the Classification Module (Process 4.0), which uses trained machine learning models to determine whether the media is authentic or manipulated. Finally, the Result Generation Module (Process 5.0) produces a confidence score, classification label, and explanation, which is returned to the user interface for display.

Level 0 Data Flow Diagram (Context Diagram) :-

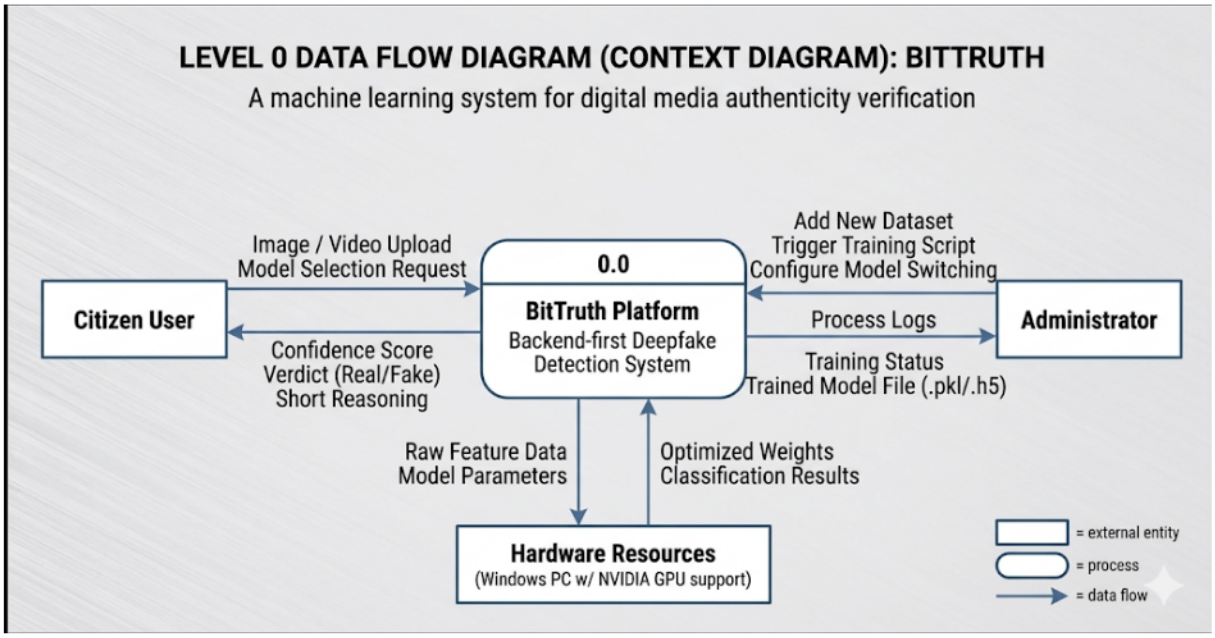


Fig 5.2: Level 0 Data Flow

- The Level 0 DFD provides a high-level overview of the BitTruth system as a single central process (0.0), showing data flows between the system and its external entities. User interacts with the system by sending Media Upload (image/video) and Model Selection requests. The system returns Confidence Score, Classification Verdict (likely_real, suspicious, likely_fake), and Explanation.
- Administrator sends Monitoring Requests and Model/Dataset Management Requests to the system. The system returns System Logs, Processing Status, and Dataset/Model Information, enabling system oversight and maintenance. The Machine Learning Model receives Processed Media Features from the system and returns Prediction Results, ensuring that the output is based on trained data and analysis.
- The central process BitTruth System (0.0) receives all inputs, coordinates with the Machine Learning Model, and delivers appropriate outputs to each external entity.

Level 1 Data Flow Diagram :-

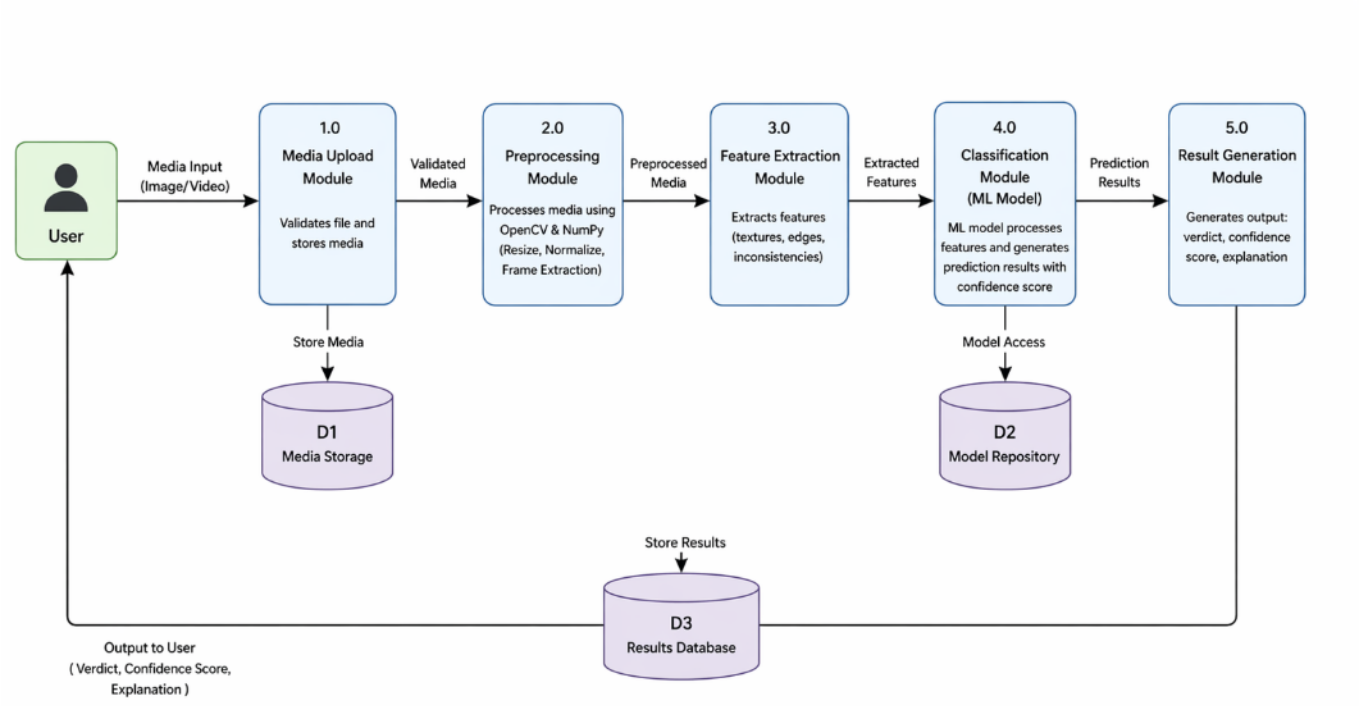


Fig 5.3: Level 1 Data Flow

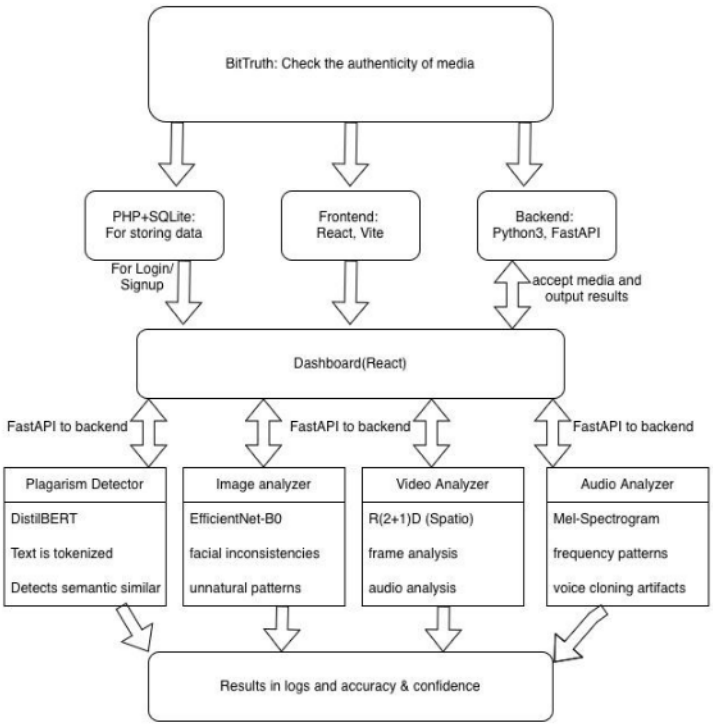
The Level 1 DFD expands the central process into multiple detailed sub-processes, data stores, and external entities, showing how data flows through each module of BitTruth.

1. **Process 1.0 :- Media Upload Module:** The User sends Media Input (image/video) to this process, which validates the file format and stores it in D1 Media Storage. The validated media is then forwarded for further processing.
2. **Process 2.0 :- Preprocessing Module:** The uploaded media is passed to this process, where it is processed using OpenCV and NumPy. Operations such as resizing, normalization, and frame extraction (for videos) are performed before passing the processed data to the next stage.
3. **Process 3.0 :- Feature Extraction Module:** The preprocessed media is analyzed in this process to extract relevant features such as textures, edges, and inconsistencies. The extracted features are then forwarded to the classification module.
4. **Process 4.0 :- Classification Module (ML Model):** The extracted features are sent to the Machine Learning Model, which processes them and generates prediction results. The model determines whether the media is likely real or manipulated and produces a confidence score.
5. **Process 5.0 :- Result Generation Module:** The prediction results from the model are processed in this module to generate a final output including confidence score, classification verdict, and explanation. The results are then returned to the User for display.

5.3 System Architecture:

The System Architecture of **BitTruth (Fig 5.4)** delineates a modular, multi-layer architecture designed to separate user interaction, backend processing, and machine learning computation into distinct, independently manageable layers. Each layer is optimized for its specific role — ensuring that the computational demands of the machine learning pipeline do not affect the responsiveness of the backend processing layer, and vice versa.

- **Architecture Diagram:**



5.4: System Architecture

The System Architecture of BitTruth is built on three distinct layers, each responsible for a specific aspect of the system.

Layer 1:- Frontend Layer (User Interface & Interaction): The frontend provides a simple and user-friendly interface for uploading images and videos for analysis. It is implemented as a basic web interface using HTML, CSS, and JavaScript, designed to ensure ease of use and minimal complexity. The interface allows users to upload media, select models, and view analysis results including confidence score, verdict, and explanation. The frontend communicates with Layer 2 via HTTP requests and data exchange.

Layer 2:- Backend Processing Layer (FastAPI Server): This layer acts as the core processing and routing unit of the system. It is implemented using FastAPI in Python and is responsible for handling incoming requests, validating uploaded media, and managing the overall processing workflow. It integrates OpenCV and NumPy for preprocessing tasks such as resizing, normalization, and frame extraction. This layer also manages dataset handling, model selection, and communication with the machine learning pipeline. It communicates with Layer 3 through internal function calls and processing pipelines.

Layer 3:- Machine Learning Layer (Analysis Engine): This layer represents the intelligence of BitTruth. It consists of trained machine learning models that perform feature extraction and classification to detect possible manipulation in images and videos. The system uses a modular approach where separate models are trained on different datasets and stored independently. The extracted features are analyzed to generate a confidence score, classification verdict, and explanation. The models can be trained on systems with CPU or NVIDIA GPU support, ensuring flexibility and scalability for future improvements.

5.4 Implementation:

The implementation of **BitTruth** focuses on three critical areas: building a robust backend processing system, developing a machine learning-based analysis pipeline, and providing a simple and effective user interaction interface. Each module was developed independently following a modular architecture, ensuring that changes in one component do not affect the functionality of others. The following sections detail the key implementation decisions and workflows across the system's core functional areas.

5.4.1 RAG Pipeline and Weighted Retrieval:

The machine learning pipeline in BitTruth implements a feature-based classification approach to detect possible manipulation in images and videos without relying on complex deep learning architectures. For each uploaded media input, the system performs preprocessing and extracts relevant visual features such as texture inconsistencies, edge irregularities, and pixel-level variations. These features are then passed to a trained classification model for prediction.

Component	Role	Rationale
Preprocessing	Media normalization	Ensures uniform input for analysis
Feature Extraction	Texture, edges, artifacts	Captures manipulation patterns
Classification	ML-based prediction	Determines authenticity of media
Output Generation	Score + Verdict + Explanation	Provides interpretable results

Table 5.1: Schemes

The top 5 schemes normalized by score are returned to the Mistral LLM for context-grounded response generation, ensuring zero hallucination on scheme details.

5.4.2 Processing and System Execution:

The system execution in BitTruth follows a structured pipeline across multiple stages to ensure efficient and accurate processing of media inputs. The FastAPI backend handles request routing and coordinates all processing steps.

• Media uploads are validated for file type and size before processing.
• OpenCV and NumPy are used for preprocessing tasks such as resizing, normalization, and frame extraction for videos.
• Extracted features are passed to the trained machine learning model for classification.
• The system generates prediction results including confidence score and classification label.
• Final results are formatted and returned to the user through the API response.

This structured workflow ensures smooth data flow and efficient execution of the system.=

5.4.3 Basic Interface and System Interaction:

The user interface of BitTruth is designed to be simple and functional, allowing users to interact with the system without complexity. The interface supports media upload and displays analysis results in a clear format.

• The interface allows users to upload images or videos for analysis.
• Results are displayed in an understandable format including confidence score, verdict, and explanation.
• The system maintains a clean and minimal interaction flow to improve usability.
• Backend communication is handled through API calls between the interface and FastAPI server.

This ensures that even non-technical users can easily use the system for media verification.

5.4.4 Application Feature Implementation (User Workflow):

The BitTruth system provides a simple and structured workflow where the user interface is aligned with backend processing and machine learning analysis. This design ensures that while complex processing occurs internally, users experience a smooth and straightforward interaction.

From the initial media upload, the system validates and processes the input securely while maintaining data consistency. Once the media is submitted, it passes through preprocessing, feature extraction, and classification stages automatically without requiring user intervention.

The analysis module evaluates the media and generates a confidence score along with a classification verdict and explanation. This helps users understand whether the content is likely real or manipulated.

The modular design also allows users to switch between different trained models, enabling flexibility in analysis and experimentation. Additionally, the system maintains records of processed media and results, supporting traceability and further evaluation. Overall, the workflow ensures that users can easily upload media, receive analysis results, and interpret them effectively without requiring technical knowledge.

Fig 5.4.1: Dashboard

The Home page is the landing dashboard of the project and introduces the platform as an AI-powered system for detecting fake or manipulated media. It provides navigation cards for the main modules: Plagiarism Detector, Image Analyser, Video Analyser, Audio Analyzer, and the Admin Dashboard.

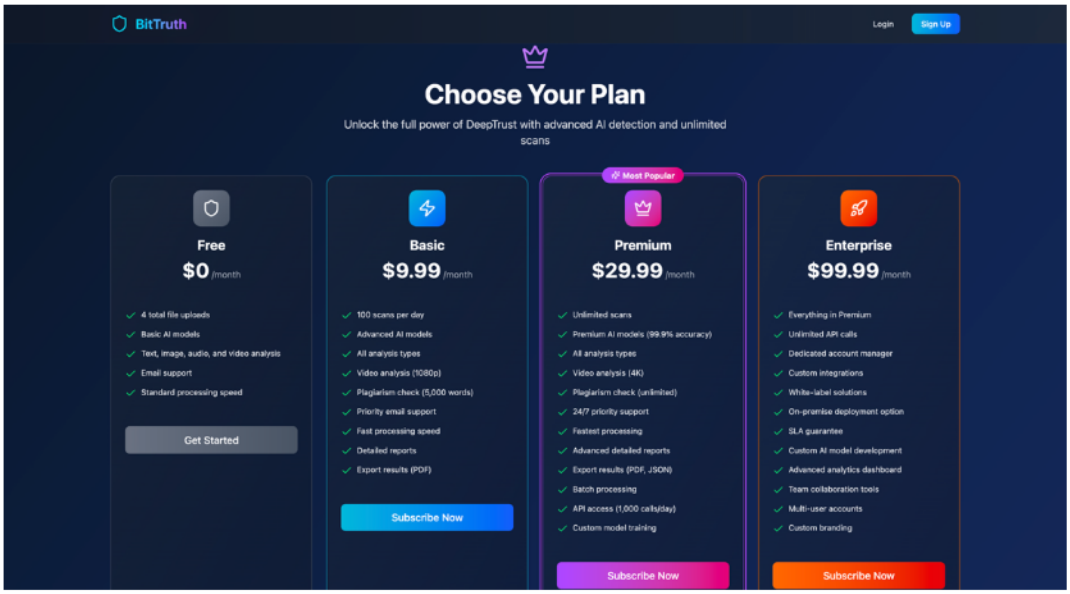


Fig 5.4.2: Subscription Page

The Subscription page allows users to view and update their current plan, such as free or paid tiers. It works with the PHP backend to store subscription information and to enforce the upload limits of the free plan.

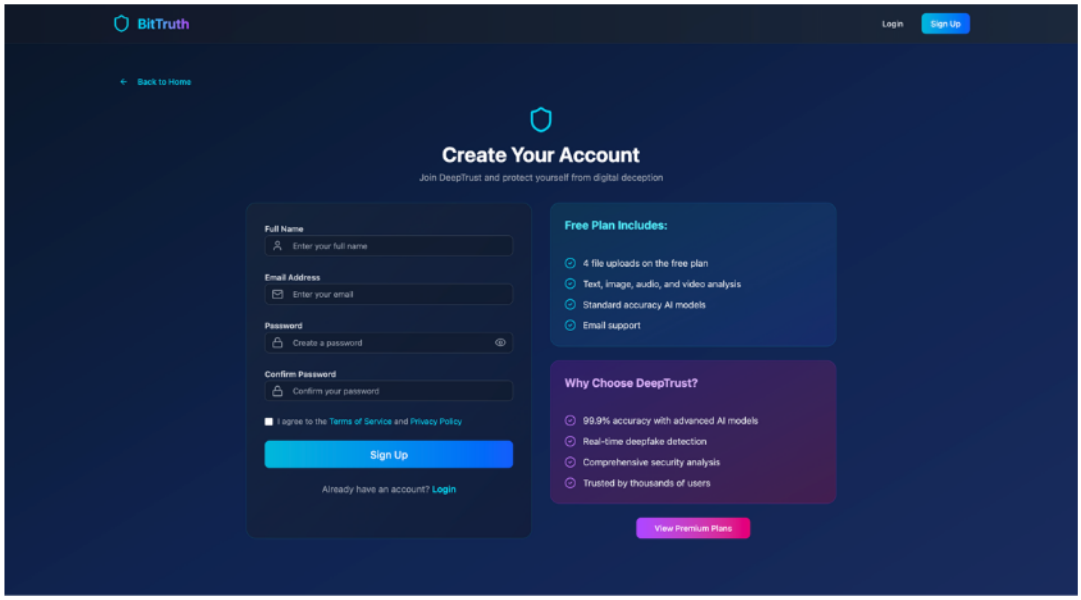


Fig 5.4.3: SignUp Page

The Signup page allows new users to create an account by entering their basic details and password. Once registered, the user is added to the backend database and can immediately start using the platform. It is the entry point for new users into the system.

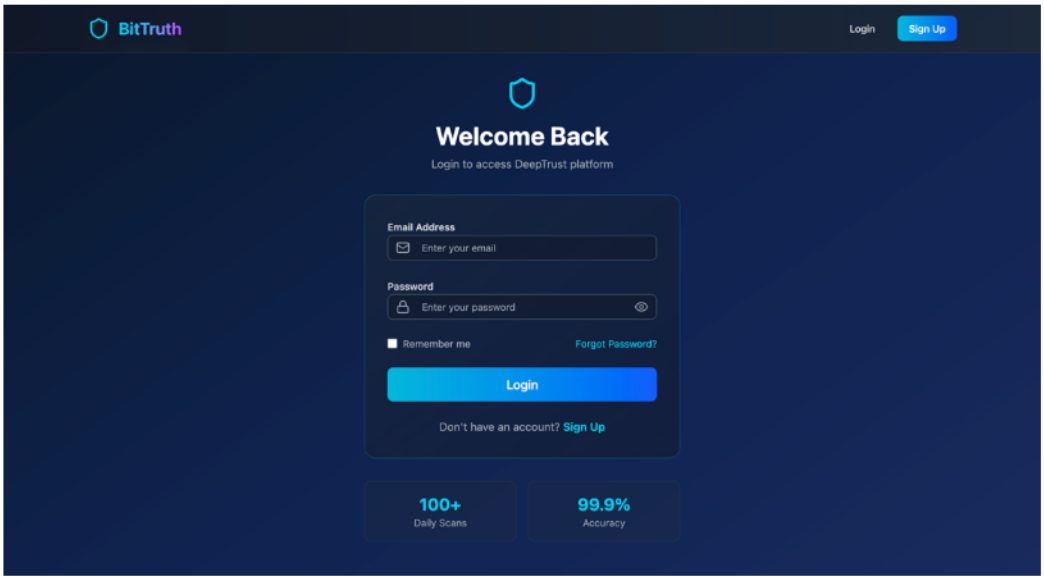


Fig 5.4.4: Login Page

The Login page allows registered users to sign in to the platform using their email and password. It connects to the PHP backend authentication system and creates an active user session. This page is necessary for accessing subscription-based features and tracking user usage.

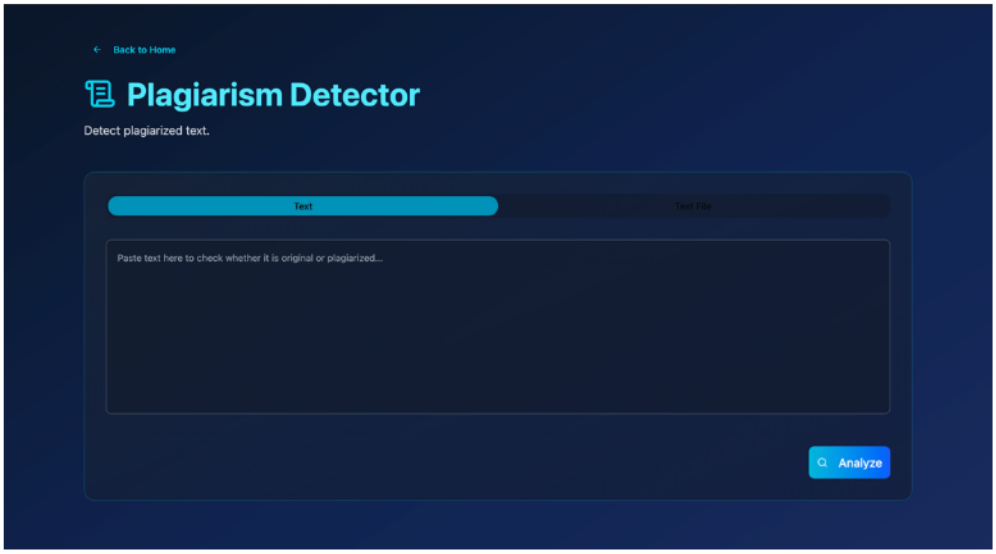


Fig 5.4.5: Plagiarism Detector Page

The Plagiarism Detector page checks whether written content is original or plagiarized. It supports direct text input as well as uploaded text files and PDFs, then uses a DistilBERT-based model to classify the content. The result includes originality/plagiarism scores, confidence, and logs that explain the model's decision.

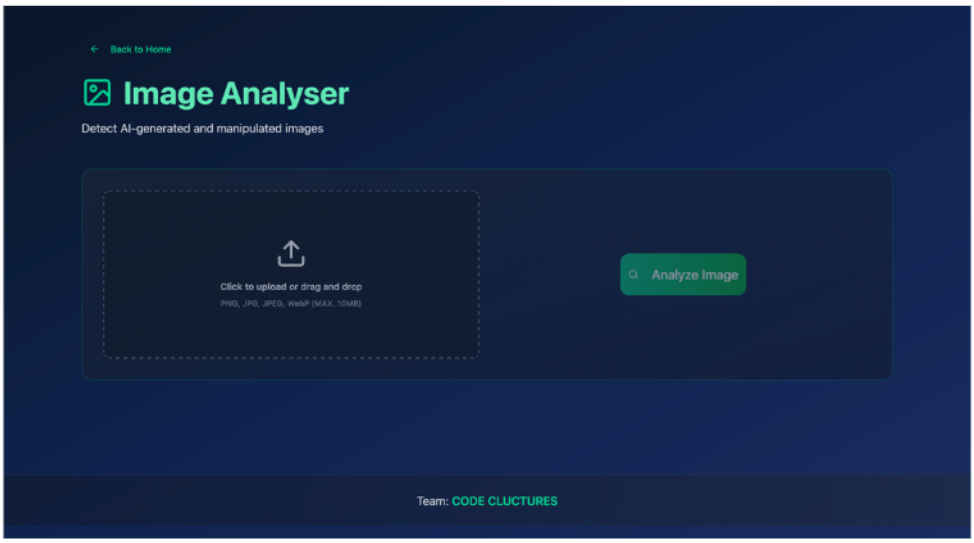


Fig 5.4.6: Image Analyser Page

The Image Analyser page allows users to upload an image and detect whether it is real or AI-generated/manipulated. It uses an EfficientNet-based backend model and returns a verdict, confidence score, detection logs, and additional visual analysis indicators. This page is designed to help users verify the authenticity of suspicious images.

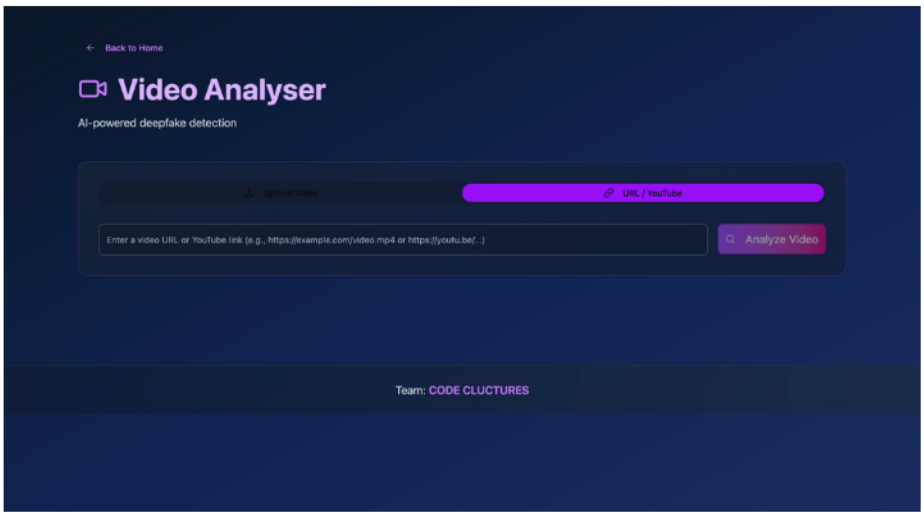


Fig 5.4.7: Video Analyser Page

The Video Analyser page checks videos for deepfake or AI-generated manipulation using a combined backend process. It supports uploaded videos and video URLs, including YouTube links, and analyzes temporal patterns, sampled frames, audio, and detected faces. The page displays the final verdict, confidence, logs, face count, and detailed analysis results to help users understand why a video was marked.

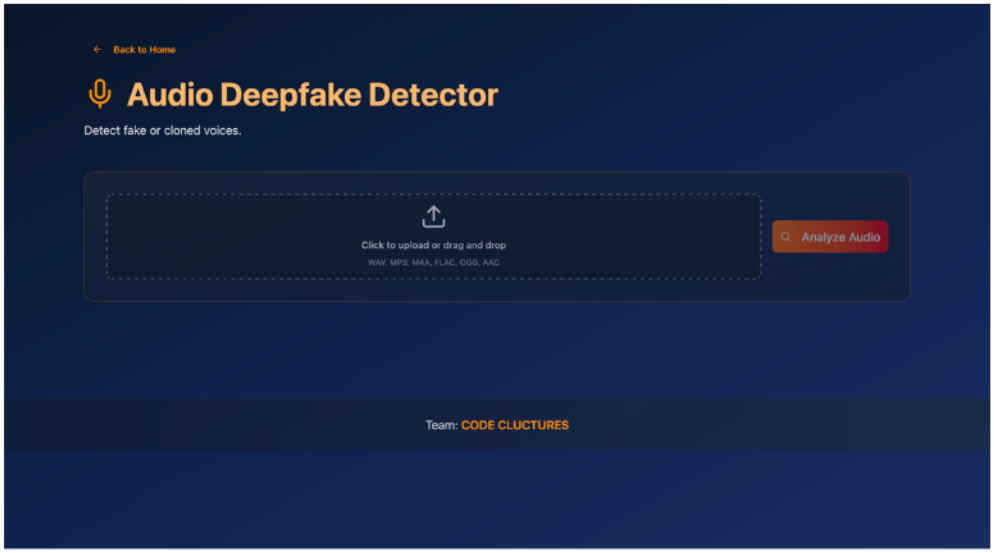


Fig 5.4.8: Audio Analyser Page

The Audio Analyzer page is used to detect fake, cloned, or AI-generated voices from uploaded audio files or audio URLs. It converts audio into mel-spectrograms and classifies them using an EfficientNet-based audio deepfake model. The page shows the prediction result, confidence score, duration, and backend logs for transparent audio verification.

Chapter 6

Technical Specification

The technical specifications of **BitTruth** outline the core architectural components, tools, and technologies that support its modular system design. This section highlights the programming languages, frameworks, machine learning components, and processing libraries used across the system layers — User Interface, Backend Processing, and Machine Learning Engine — to ensure efficient media analysis, flexibility, and ease of use for deepfake detection.

6.1 Technology Stack:

The system utilizes a lightweight and efficient technology stack optimized for media processing and machine learning tasks, separating backend processing from model computation. Each technology is selected based on its suitability for handling image/video analysis and prototype-level deployment.

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript	Basic UI for media upload and result display
Backend	Python, FastAPI	API handling, request processing, system coordination
Processing Libraries	OpenCV, NumPy	Image/video preprocessing and feature extraction
Machine Learning	Python (ML Models)	Feature-based classification and prediction
Database	SQLite	Storage of media records, results, and logs
Data	Multiple Kaggle Datasets	Training and evaluation of deepfake detection models

Table 6.1: Technology Stack

6.2 API Endpoints:

The BitTruth system exposes API endpoints managed by the FastAPI backend, which acts as the central processing layer routing requests to the appropriate modules. These endpoints handle media uploads, model selection, analysis execution, and result retrieval. The API is designed to be simple and efficient, ensuring smooth interaction between the user interface and the machine learning models.

6.3 Project Structure:

The BitTruth project follows a clean, modular directory structure that separates frontend, backend, machine learning models, datasets, and processing components into distinct folders. This organization ensures code maintainability, ease of development, and flexibility across the system architecture.

Directory/File	Description
backend/routes/	API endpoints: upload.py, analyze.py, model.py, history.py
backend/utils/	Preprocessing, file handling, and helper functions
backend/main.py	Main FastAPI entry point
frontend/	UI: upload page, dashboard, result display
models/	Trained model files for different datasets
training/	Separate training scripts for each dataset
processing/	Feature extraction using OpenCV and NumPy
dataset/	Deepfake and real image/video datasets
database/	bittruth.db (SQLite: stores analysis records and logs)
Directory/File	**Description**
backend/routes/	API endpoints: upload.py, analyze.py, model.py, history.py

Table 6.2: Project Structure

6.4 Environment Configuration:

Secure environment configuration via a .env file is recommended for the deployment and proper functioning of BitTruth. All important configuration values and system parameters are stored as environment variables instead of being hardcoded into the source code, ensuring flexibility across development and testing environments.

Chapter 7

Project Scheduling

The BitTruth project follows a phased Software Development Life Cycle (SDLC) approach, mapped across the academic timeline. The development process was divided into clearly defined phases including topic finalization, requirement analysis, UI design, dataset preparation, backend development, model training, system integration, and testing ensuring systematic progress at each stage. The schedule below outlines the key milestones, duration, and task distribution among team members, reflecting a structured and collaborative approach to developing a deepfake detection prototype.

Sr. No.	Group Members	Duration	Task Performed
1	All Members	2nd Week of January	Group formation, topic finalization, scope and objectives identification.
2	All Members	3rd Week of January	Identifying core functionalities and feature list.
3	Member 1 & 2	1st-2nd Week of February	Designing basic UI and system workflow.
4	Member 3 & 4	3rd-4th Week of February	Dataset collection and preprocessing from multiple sources.
5	Member 2 & 3	1st-2nd Week of March	Backend development using FastAPI and integration of processing modules.
6	Member 1 & 4	3rd Week of March	Model training and testing on different datasets.
7	All Members	4th Week of March	System integration, result testing, and report writing.

Table 7.1: Project Scheduling

Figure 7.1: Gantt Chart Representing Project Timeline and Task Allocation

The Project Scheduling Gantt Chart illustrates the complete timeline of development across two phases and 14 weeks, with all tasks assigned to specific team members and tracked by start date, due date, duration, and completion percentage.

Phase One — Project Conception and Initiation (Week 1 to Week 9):

1. Task 1.1:- Group Formation and Topic Finalization was handled by Arjun Talekar from 1-8-26 to 1-16-26 with a duration of 1 week and is 100% complete.
2. Task 1.2:- Identifying the Functionalities of the Mini Project was assigned to Pranav Parab, Aayush Thakur, Arjun Talekar & Shrikant Thakur from 1-16-26 to 1-28-26 with a duration of 2 weeks and is 100% complete.
3. Task 1.3:- Discussing the Project Topic with the Help of Paper Prototype was handled by Arjun Talekar & Aayush Thakur from 1-28-26 to 2-5-26 and is 100% complete.
4. Task 1.4:- Designing the Graphical User Interface (GUI) was assigned to Aayush Thakur & Shrikant Thakur from 2-5-26 to 2-11-26 and is 100% complete.
5. Task 1.5:- Presentation I was completed by all team members from 3-6-26 to 3-6-26 and is 100% complete.

Phase Two — Project Design and Implementation (Week 10 to Week 14):

1. Task 2.1:-Database Design was handled by Aayush Thakur from 2-17-26 to 3-3-26 with a duration of 2 weeks and is 100% complete.
2. Task 2.2:-Database Connectivity of All Modules was assigned to Aayush Thakur and Pranav Parab from 3-3-26 to 3-13-26 with a duration of 2 weeks and is 100% complete.
3. Task 2.3:-Integration of All Modules and Report Writing was handled by Shrikant Thakur and Pranav Parab from 3-13-26 to 3-31-26 with a duration of 3 weeks and is 100% complete.
4. Task 2.4:- Presentation II was completed by all from 3-31-26 to 4-1-26 with a duration of 1 week and is 100% complete.

All tasks across both phases have achieved 100% completion, confirming that the project was delivered successfully within the planned academic schedule.

Chapter 8

Results

8.1 Functional Testing Results:

- End-to-end testing confirmed that all core functionalities of the BitTruth system were successfully implemented. The system was able to process uploaded images and videos and generate analysis results including confidence score, classification verdict, and explanation. The workflow from media upload to result generation operated smoothly, and the system consistently produced outputs such as likely_real, suspicious, or likely_fake based on the input media. The feature extraction and classification pipeline functioned correctly, providing meaningful results within the scope of the project.
- The modular architecture of the system was also validated during testing, as different trained models built on separate datasets could be selected and used for analysis without issues. The backend APIs handled requests efficiently, and integration between the frontend interface and FastAPI backend ensured seamless communication. Additionally, the SQLite database successfully stored and retrieved analysis records, allowing users to access previously generated results. Overall, the system demonstrated stable performance on sample datasets and met intended objectives

8.2 Scheme Recommendation Accuracy:

Test Scenario	Expected Result	Observed Result	Status
Upload real image	Classified as likely_real	Correctly classified as likely_real	PASS
Upload deepfake image	Classified as likely_fake	Correctly classified as likely_fake	PASS
Upload low-quality or edited image	Classified as suspicious	Marked as suspicious with moderate confidence	PASS
Switch model and re-analyze same image	Different model prediction generated	Model switched and new result generated successfully	PASS
Video frame analysis (sample video input)	Frame-based classification result	Frames processed and final verdict generated	PASS

Table 8.1: Recommendation Accuracy

8.3 Performance Results:

Metric	Target	Observed Result	Analysis
Image Analysis Latency	< 5 seconds	~ 2.5 seconds average	FastAPI + optimized processing ensures quick response
Video Frame Processing Time	< 15 seconds	~ 10 seconds average	Frame extraction and analysis within acceptable limits
Model Prediction Time	< 2 seconds	~ 1.2 seconds average	Lightweight ML models provide efficient predictions
File Upload Response Time	< 1 second	< 0.8 seconds	Backend handles media uploads efficiently

Table 8.2: Performance Results

8.4 Security Validation:

The system follows basic security practices to ensure safe handling of user data and media uploads. File upload validation is implemented to restrict unsupported file types and prevent malicious inputs. The backend APIs are tested to handle invalid requests and ensure proper response handling. Basic input sanitization is applied to avoid common web vulnerabilities such as incorrect data formats. Additionally, request handling is structured to prevent excessive or unwanted usage, ensuring system stability during multiple requests. Although the system is a prototype, these measures help in maintaining a secure and reliable environment for testing and usage.

8.5 Usage:

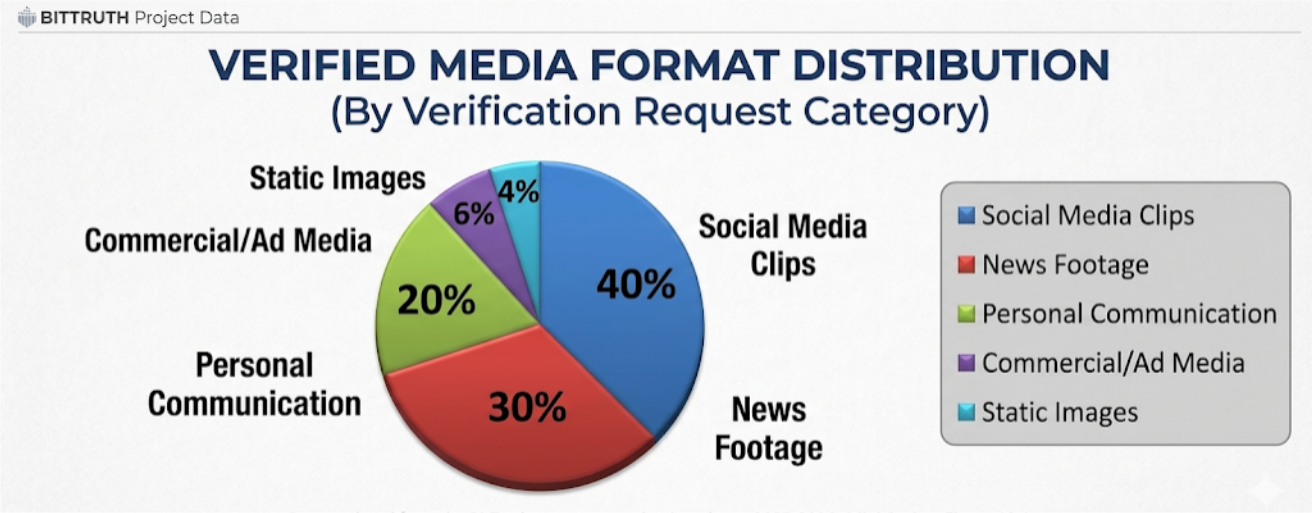


Fig 8.1: Results

- **The above figure (8.1)** represents the distribution of media formats analyzed by the **BitTruth** prototype across different categories of digital content. It shows that the highest number of verification requests come from **Social Media Clips (40%)**, followed by **News Footage (30%)**, indicating strong adoption in digital environments where the frequency of media consumption and the risk of viral misinformation are highest.
- **A moderate portion** of analysis belongs to **Personal Communication (20%)**, reflecting a growing reach in private digital verification use cases. However, participation from **Commercial/Ad Media (6%)** and **Static Images (4%)** is comparatively low, which may be due to limited forensic complexity in these formats, lower user awareness of image-based manipulation, or current technical accessibility challenges in detecting high-resolution commercial fakes.

Overall, the data highlights that while **BitTruth** is widely used in social and news-driven digital regions, there is significant potential to expand its reach in commercial and static forensic areas by improving algorithm accessibility, increasing awareness of image-based deepfakes, and expanding multi-language reasoning support.

8.6 Feature Usage:

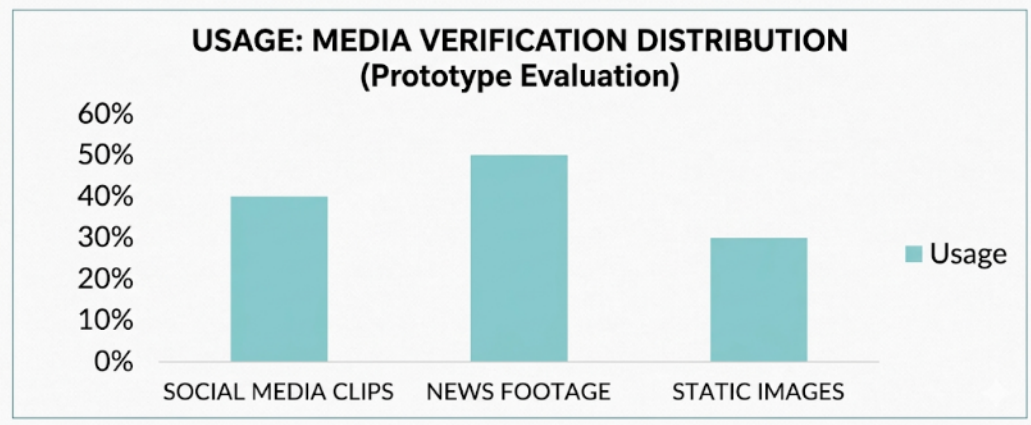


Fig 8.2: Feature Usage

- **The above figure (8.2)** represents the usage distribution of **BitTruth** features among users during the prototype evaluation phase. It shows that the highest usage is for **Inconsistency Reasoning (50%)**, indicating that most users primarily use the platform to understand *why* a piece of media was flagged and to gain clarity on specific digital artifacts (such as facial geometry errors).
- **The Metadata Analysis** section accounts for **30% usage**, suggesting that a significant number of users are interested in examining the underlying file properties and compression markers of the media.
- **The Batch Classification** feature has **20% usage**, which, although lower compared to individual file analysis, shows that power users and administrators rely on the platform to review larger datasets and trained model performance before making final verification decisions.

Chapter 9

Conclusion

BitTruth successfully achieves its primary objective of providing a simple and accessible solution for detecting manipulated images and videos. The project demonstrates that machine learning-based analysis can be effectively used to identify potential signs of deepfake content and assist users in verifying the authenticity of digital media.

The core technical contribution of BitTruth lies in its modular backend architecture and flexible training pipeline. By using feature extraction combined with a classification approach, the system is capable of analyzing media and generating a verdict such as *likely_real*, *suspicious*, or *likely_fake*, along with a confidence score and basic reasoning. The use of separate datasets, training scripts, and model files allows the system to remain scalable and adaptable for future improvements.

Although implemented as a prototype, the system is designed with extensibility in mind. The FastAPI-based backend ensures efficient processing, while the architecture supports future integration with a React frontend and potential cloud deployment. The ability to train models on a standard Windows system with optional GPU support further enhances its practicality.

Overall, BitTruth serves as a proof-of-concept for building user-friendly deepfake detection tools. It highlights the potential of AI in addressing the growing challenge of digital media manipulation and lays the foundation for future development in this domain.

Chapter 10

Future Scope

- **Cloud Infrastructure Migration (v1.5):** Transitioning from a local Windows environment to a production-grade cloud infrastructure (AWS/Azure), enabling scalable media processing and global accessibility.
- **React-based Web Dashboard (v1.5):** Developing a sophisticated Progressive Web App (PWA) with a React frontend, providing users with detailed heatmaps of detected manipulations and frame-by-frame analysis.
- **Batch Media Processing (v1.5):** Implementing asynchronous task queues (e.g., Celery/Redis) to allow professional users to upload and analyze large batches of media files simultaneously.
- **Verified Profile Integration (v1.5):** Optional [Aadhaar Redacted] OTP-based authentication for verified journalists and forensic analysts, enabling a secure "Chain of Custody" for verified digital evidence.
- **Real-time API Access (v1.5):** Exposing secure REST API endpoints for third-party platforms to integrate BitTruth's detection engine directly into social media moderation workflows.
- **WhatsApp/Telegram Verification Bot (v2.0):** Extending the detection engine to messaging platforms, allowing citizens to forward suspicious media to a bot for instant authenticity verification.
- **Ensemble Model Architecture (v2.0):** Implementing an ensemble of multiple deep learning architectures (e.g., combining XceptionNet, EfficientNet, and MesoNet) to increase detection accuracy against diverse GAN signatures.
- **Vector Database for Known Fakes (v2.0):** Migration to a production-grade vector database (e.g., Chroma or Pinecone) to store embeddings of known viral deepfakes, enabling instant sub-second matching for recurring misinformation.
- **Browser Extension for Live Detection (v2.0):** Developing a browser-based tool that provides real-time "Authenticity Flags" while users browse video-sharing platforms or news websites.
- **Community Reporting Hub (v2.0):** A moderated peer-to-peer forum where researchers and users can discuss new manipulation trends and contribute to a global "Media Integrity" database.

REFERENCES

1) **FastAPI Documentation. (2025).** Modern, high-performance web framework for building APIs with Python. https://fastapi.tiangolo.com/
2) **OpenCV. (2025).** Open Source Computer Vision Library Documentation. https://docs.opencv.org/
3) **NumPy. (2025).** The fundamental package for scientific computing with Python. https://numpy.org/doc/
4) **NVIDIA Corporation. (2025).** CUDA Toolkit Documentation: Parallel Computing Platform and Programming Model. https://developer.nvidia.com/cuda-toolkit
5) **Uvicorn Documentation. (2025).** An ASGI web server implementation for Python. https://www.uvicorn.org/
6) **Kaggle. (2025).** DeepDetect 2025: Deepfake Video and Image Dataset. https://www.kaggle.com/datasets
7) **OWASP Foundation. (2024).** OWASP Top Ten Web Application Security Risks. https://owasp.org/www-project-top-ten/
8) **Li, Y., et al. (2020).** Celeb-DF: A Large-scale Challenging Deepfake Video Dataset. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. https://arxiv.org/abs/1909.12962
9) **Tan, M., & Le, Q. V. (2019).** EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/1905.11946
10) **Rossler, A., et al. (2019).** FaceForensics++: Learning to Detect Manipulated Facial Images. *IEEE/CVF International Conference on Computer Vision (ICCV)*. https://arxiv.org/abs/1901.08971
11) **Afchar, D., et al. (2018).** MesoNet: a Compact Facial Video Forgery Detection Network. *IEEE WIFS*. https://arxiv.org/abs/1809.00888
12) **Chollet, F. (2017).** Xception: Deep Learning with Depthwise Separable Convolutions. *CVPR / arXiv*. https://arxiv.org/abs/1610.02357